Command line demonstration

The same investigation as docs/demo.md, without the window. This is the fallback if the GUI cannot run on the demonstration machine. Every command and every output below was run on the repository as committed. Paths are relative to the repository root.

Set the environment once:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
export QT_QPA_PLATFORM=offscreen
```

The last line keeps Qt quiet even though none of these commands open a window.

1. Full pipeline: PCAP to project to report

```
python -m src.project.cli pipeline \
  --pcap tests/corpus/corpus_capture_01.pcapng \
  --project /tmp/demo.madrigal \
  --rule examples/corpus_rule_v2.json \
  --report /tmp/demo_report.md
```

Expected: one JSON summary line on stdout.

```
{"project": "/tmp/demo.madrigal", "normalized": ".../results/normalized.json",
 "sessions": 3, "streaming": false, "result": ".../results/result.json",
 "report": ".../reports/REPORT.md", "report_html": ".../reports/REPORT.html"}
```

The project directory holds the capture, the normalized JSON, the rule and the rendered report; /tmp/demo_report.md is the same report written where it was asked for. Open reports/REPORT.html in a browser: it is self-contained and needs no network.

2. Apply rule v1 to one direction

```
python -m src.protocol.cli apply \
  --rule examples/corpus_rule_v1.json \
  --capture tests/corpus/reference_export/corpus_capture_01.normalized.json \
  --out /tmp/result_v1.json
```

Expected summary on stderr:

```
/tmp/result_v1.json: rule corpus_request v1, 60 matched, 0 mismatched
```

apply frames the first session, direction A_to_B. The written result is a document that validates against docs/schemas/result.schema.json; every message carries its offset, length, decoded fields, status and the provenance_range that ties it to packets.

3. Verify rule v1 over the whole capture

```
python -m src.protocol.cli verify \
  --rule examples/corpus_rule_v1.json \
  --capture tests/corpus/reference_export/corpus_capture_01.normalized.json \
  --out /tmp/report_v1.json
```

Expected summary on stderr:

```
rule corpus_request v1: 280 messages, 80 counterexamples
```

The report counts every message in every direction with the six core statuses:

```
"counts": {"matched": 60, "mismatched": 80, "not_applicable": 140, ...}
```

The 80 `mismatched` are the counterexamples: requests whose command byte is `2` or `3`, which v1 forbids. Each is listed in `contradictions` with its session, direction, offset and bytes.

4. Apply rule v2 and re-verify

```
python -m src.protocol.cli verify \
  --rule examples/corpus_rule_v2.json \
  --capture tests/corpus/reference_export/corpus_capture_01.normalized.json \
  --out /tmp/report_v2.json
```

Expected summary on stderr:

```
rule corpus_request v2: 280 messages, 0 counterexamples
```

counts: `matched=140`, `mismatched=0`, `not_applicable=140`. The refinement widened `command.expected` from `[1]` to `[1, 2, 3]` and nothing else changed.

5. Diff the two rule versions

```
python -m src.protocol.cli diff \
  --rule-a examples/corpus_rule_v1.json \
  --rule-b examples/corpus_rule_v2.json
```

Expected summary (stderr), first lines:

```
rule corpus_request: v1 -> v2
  ~ field command.expected: [1] -> [1, 2, 3]
```

The JSON body separates the rule diff (`changed_fields`) from any report diff when `--report-a` and `--report-b` are given.

6. Correlate requests with the action journal

```
python -m src.protocol.cli correlate \
  --rule examples/corpus_rule_v2.json \
  --capture tests/corpus/reference_export/corpus_capture_01.normalized.json \
  --journal tests/corpus/corpus_journal.md \
  --out /tmp/correlation.json
```

Expected summary inside `/tmp/correlation.json`:

```
"summary": "60 of 60 messages correlated, 125 journal entries unmatched"
```

Against the full capture (all directions), the investigation script reports the same agreement at higher volume: 140 of 140 requests on `corpus_capture_01` and 40 of 40 on `corpus_capture_02`, with the decoded `target` equal to the journal parameter on every correlated request.

7. Quality metrics

```
python -m src.protocol.cli metrics --report /tmp/report_v2.json
```

Expected summary on stderr:

```
rule corpus_request v2: coverage 0.500, precision 1.000, counterexample density 0.0/100
```

Coverage is 0.5 because the rule scopes `A_to_B` and half the messages are responses. Precision is 1.0 because no matched message is contradicted.

8. Export the rule as a parser

```
python -m src.protocol.cli export \
  --rule examples/corpus_rule_v2.json --format kaitai --out /tmp/parsers/
python -m src.protocol.cli export \
  --rule examples/corpus_rule_v2.json --format python --out /tmp/parsers/
```

Expected: two files under `/tmp/parsers` (`corpus_request_v2.ksy` and `corpus_request_v2.py`), a Kaitai Struct description and a standalone Python module that repeats the engine's framing. The trailing separator makes `--out` a directory; without it the path is taken as the output file and the two exports overwrite each other. Both are generated from the rule, not written by hand.

9. Regenerate the investigation write-up

```
python -m scripts.run_reference_investigation
```

Expected: `docs/REFERENCE_INVESTIGATION.md` is rewritten from the captures with its framing, counterexample, refinement, transfer, journal and alternatives sections. The command fails if the corpus is missing rather than inventing data.

10. Alternative readings of a hypothesis field

```
python -m src.protocol.cli alternatives \
  --rule examples/corpus_rule_v2.json \
  --report /tmp/report_v2.json \
  --corpus tests/corpus/reference_export/corpus_capture_01.normalized.json \
  --all
```

Expected summary inside the JSON (one block per `hypothesis` field):

```
"field_name": "command", "declared_meaning": "command", "best": "entropy_enum"
  entropy_enum   support 140  contradict 0  score 1.00
  low_cardinality support 140 contradict 0  score 1.00
  constant       support 60   contradict 80 score 0.43
```

`constant` is the reading rule v1 declared; on this capture most messages contradict it, while `entropy_enum` fits all of them. That is the evidence behind the refinement, and it is produced by the tool, not asserted. The command prints seven alternatives for `command` and ranks `entropy_enum` first; `--all` reports every hypothesis field at once, while `--field command` limits the output to one field.

11. Move the project and reopen it

```
python -m src.project.cli export --project /tmp/demo.madrigal --out /tmp/demo.zip
python -m src.project.cli import --in /tmp/demo.zip --target /tmp/demo.moved
```

The returned JSON is `{"archive": "/tmp/demo.zip"}`; the import returns `{"project": "/tmp/demo.moved", "broken": []}` where an empty `broken` list means every capture matched its recorded digest. Reopen the moved project with:

```
python -m src.project.cli open --path /tmp/demo.moved --show
```

Expected: the project manifest, with the capture registered under `captures/corpus_capture_01.pcapng` and its sha256. A digest mismatch aborts the import rather than importing a changed file.

Timing

step	minutes
1 pipeline	2
2-4 apply and verify	3
5 diff	1
6 correlate	1
7 metrics	1
8 export	1
9 investigation	1
10 project move	1
total	11